

What will we be covering in this session:

1. Conditions (5%)
2. Loops (30%)
 - a. Specifically Patterns
3. Functions (15%)
4. Arrays (50%)

Conditions:

1:

```
int main()
{
    int a;
    if(a=1)
        cout << "Hello world!";
    Else if (a = 2)
        cout << "Ok World!";
    return 0;
}
a = ? , Output = ?
```

2:

```
int main()
{
    if (0 || 1 && 1 && 1 && 0)
        cout << "Hello world!";
    return 0;
}
```

3:

```
int main()
{
    if (0 || 1 && 1 && 1 && 0)
        cout << "Hello world!";

    return 0;
}
```

Loops:

a)

```
Int main()
{
    Int i;0
    for(;j<5;j++)
    {
        Cout << "Hello world!"
    }

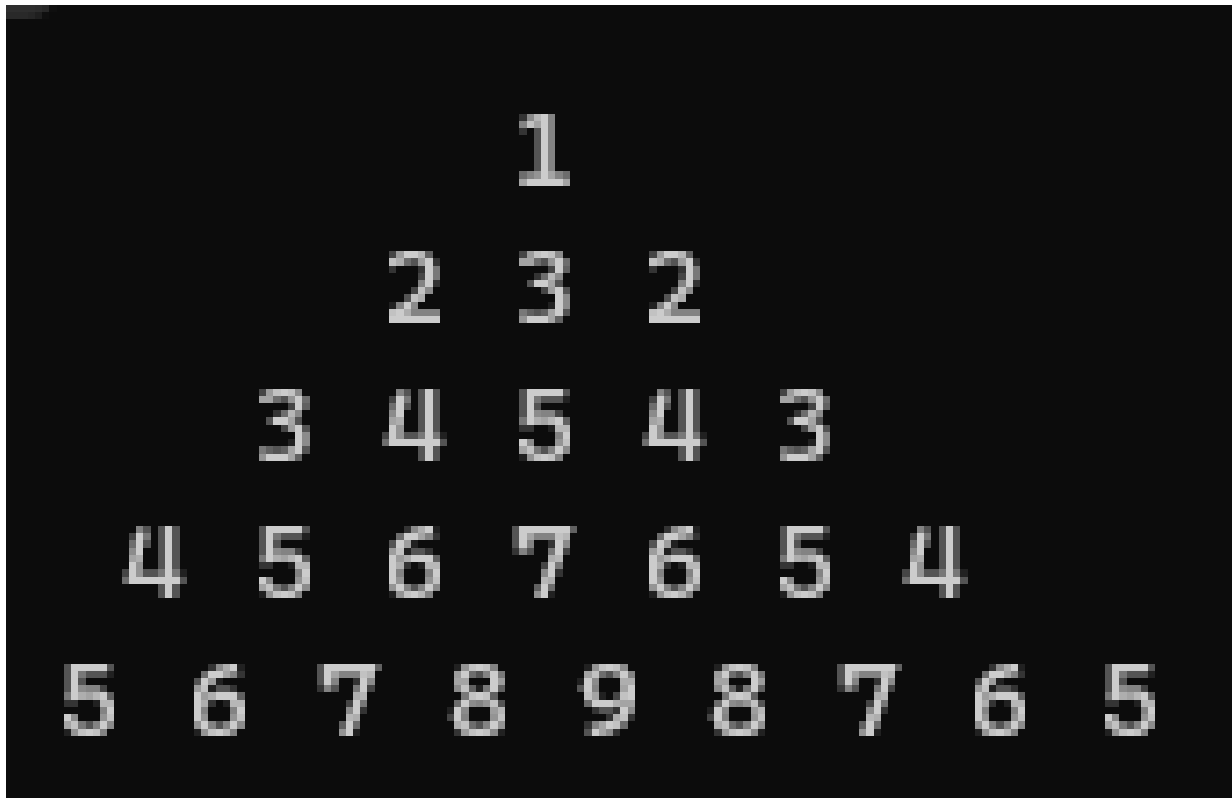
    Return 0;
}
```

b)

```
int main()
{
    int j=10;
    for (; j; j--)
    {
        cout << "Hello world!\n";
    }

    return 0;
}
```

Pattern1:



Solution:

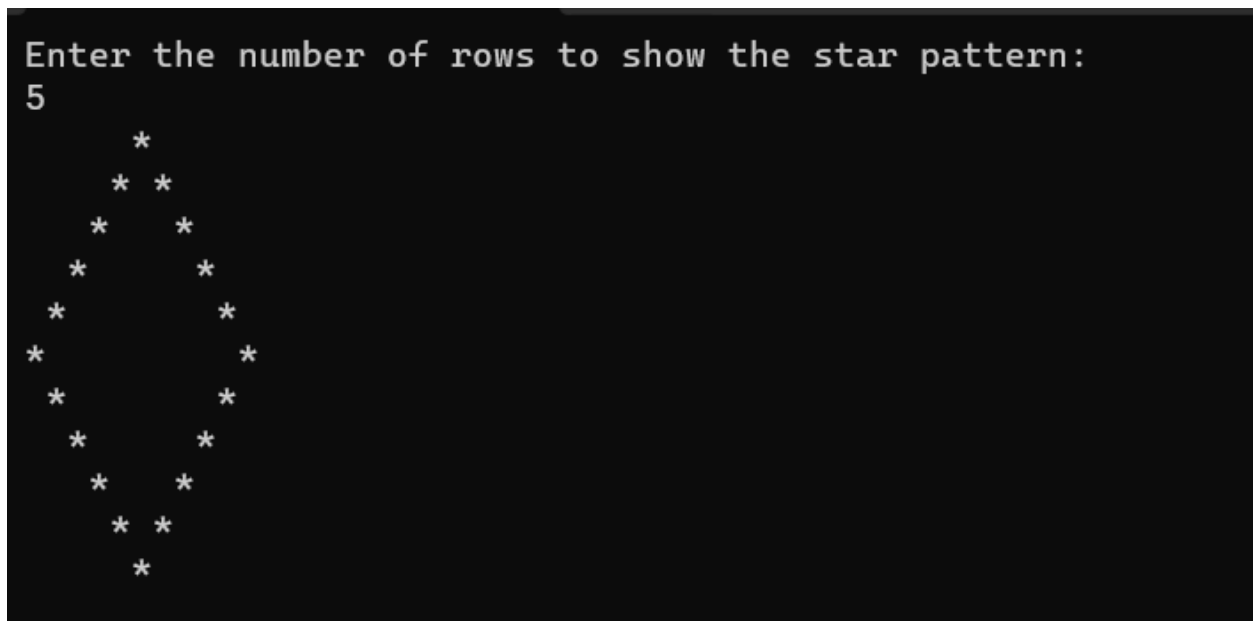
```
// ***** Pattern 1 *****  
int i = 0, j = 0, range = 5, k = 0, count = 0, count1 = 0;  
    for (i = 1; i <= range; ++i)  
    {  
        for (j = 1; j <= range - i; ++j)  
        {  
            cout << " ";  
            ++count;  
        }  
        for (j = 1; j <= (range - 1) - i; ++j)  
        {  
            cout << " ";  
        }  
        while (k != 2 * i - 1)  
        {  
            if (count <= range - 1)
```

```

        {
            cout << i + k << " ";
            ++count;
        }
        else
        {
            ++count1;
            cout << i + k - 2 * count1 << " ";
        }
        ++k;
    }
    count1 = count = k = 0;
    cout << "\n";
}

```

Pattern 2:



```

cout << "Enter the number of rows to show the star pattern: ";
int n, x, y, s = 1, k;
cin >> n;
for(x = 0; x <= n; x++)
{
    for(y = n; y > x; y--)
    {

```

```

cout << " ";
}
cout << "*";
if (x > 0)
{
for(k = 1; k <= s; k++)
{
cout << " ";
}
s += 2;
cout << "*";
}
cout << "\n";
}
s -= 4;
for(x = 0; x <= n -1; x++)
{
for(y = 0; y <= x; y++)
{
cout << " ";
}
cout << "*";
for(k = 1; k <= s; k++)
{
cout << " ";
}
s -= 2;
if(x != n -1)
{
cout << "*";
}
//ending line after each row
cout << "\n";
}

```

Pattern 3:

Output ?

```

int main()
{
    int m = 7, n = m / 2 + 1;

```

```

char ch = '*';
for (int i = 0; i < n - 1; i++)
{
    for (int j = 0; j < i; j++)
    {
        cout << " ";
    }
    for (int j = 0; j < m; j++)
    {
        if (j == m - 1 || j == 0)
            cout << ch;
        else
            cout << " ";
    }
    m -= 2;
    cout << endl;
}

return 0;
}

```

Pattern 4:

Output ?

```

int main()
{
    int n;
    cout << "Enter a number: " << endl;
    cin >> n;
    int x = 2 * n - 1;
    int inner_space = 1;
    int k = n - 1;
    for (int i = 1; i <= n; i++)
    {
        if (i == 1) {

            for (int i = 1; i <= x; i++)
            {
                cout << "*";
            }
            cout << endl;
        }
    }
}

```

```

    if (i > 1) {

        for (int j = 1; j <= k; j++) {
            cout << "*";
        }
        for (int j = 1; j <= inner_space; j++) {
            cout << " ";
        }

        for (int j = 1; j <= k; j++) {
            cout << "*";
        }
        inner_space += 2;
        --k;
        cout << endl;
    }

}

return 0;
}

```

Find HCF/GCD

```

#include <iostream>
using namespace std;

int main() {
    int n1, n2, hcf;
    cout << "Enter two numbers: ";
    cin >> n1 >> n2;

    // swapping variables n1 and n2 if n2 is greater than n1.
    if ( n2 > n1) {
        int temp = n2;
        n2 = n1;
        n1 = temp;
    }

    for (int i = 1; i <= n2; ++i) {
        if (n1 % i == 0 && n2 % i == 0) {

```

```

        hcf = i;
    }
}

cout << "HCF = " << hcf;

return 0;
}

```

ARRAYS

ARRAYS 0

```

int nums[] = { 1,2,3,4,5 };

for (int i = 0; i < 5; i++)
{
    cout << *(nums + i) << " ";
}

```

ARRAYS 1:

BUBBLE SORT

```

int main()
{
    int arr[] = { 16, 2, 1, 6, 0, 8 };
    int n = sizeof(arr) / sizeof(arr[0]);

    int i, j;
    bool swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = false;
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
            }
        }
    }
}

```



```

        arr[j + 1] = temp;
        swapped = true;
    }
}

//if (swapped == false)
//    break;
}

for (int j = 0; j < n; j++) {
    cout << arr[j] << " ";
}

return 0;
}

```

ARRAYS 2

Write a C++ program that takes an array of characters as input and reorders the elements in such a way that lowercase letters are moved to the beginning of the array while keeping the original order of other characters intact.

Solution:

```

#include <iostream>
using namespace std;

void processNum(char nums[], int n) {
    int i = -1, j = 0;
    for (; j < n; ++j)
    {
        if (nums[j] >= 'a' && nums[j] <= 'z')
        {
            int temp = nums[i + 1];
            nums[i + 1] = nums[j];
            nums[j] = temp;
            ++i;
        }
    }
}

```

```

    }
}

int main() {
    char nums[] = { 'A', 'a', 'B', 'c', 'd', 'C', 'D', 'b' };

    int n = 8;

    cout << "Original Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << nums[i] << " ";

    }
    processNum(nums, n);

    cout << "\nProcessed array : ";
    for (int i = 0; i < n; i++)
    {
        cout << nums[i] << " ";

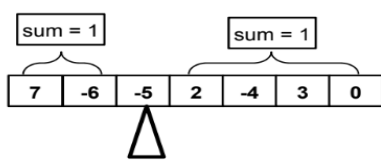
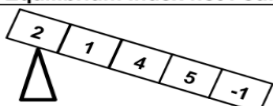
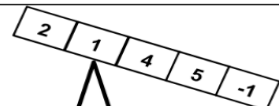
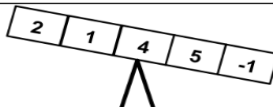
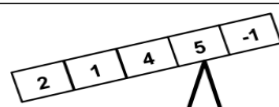
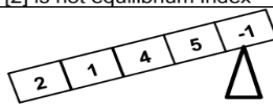
    }

    return 0;
}

```

Array 3:

Q4. [10] Write a C++ function called **findEquilibrium** that accepts an integer array A of size n and finds the *equilibrium index* of the array if it exists. An index k of an array is said to be an equilibrium index if the sum of all the elements on the left of k is equal to the sum of all the elements on the right of k . Precisely, k is an equilibrium index of A if and only if $\sum_{i=0}^{k-1} A[i] = \sum_{i=k+1}^{n-1} A[i]$. For clarity, consider this example below. If the index is not found, the function returns -1.

Equilibrium Index Found	Equilibrium Index not Found	
	 [0] is not equilibrium index	 [1] is not equilibrium index
	 [2] is not equilibrium index	 [3] is not equilibrium index
	 [4] is not equilibrium index	
findEquilibrium returns 2 since equilibrium found at index number 2.	findEquilibrium returns -1 since equilibrium not found at any index 0, 1, 2, 3 and 4.	

Solution:

```
int main() {
    int str[] = { 7,-6,-5,2,-4,3,0};

    int length = sizeof(str)/sizeof(str[0]);
    for (int i = 0; i < length; i++)
    {
        int leftSum = 0, rightSum = 0;

        for (int j = 0; j < i; j++)
        {
            leftSum += str[j];
        }

        for (int j = i+1; j < length; j++)
        {
            rightSum += str[j];
        }

        cout << "str[i] = " << str[i] << ", left sum = " << leftSum << ", right sum = " << rightSum <<
endl;
    }

    return 0;
}
```

Array 4:

Write a C++ function which takes 1-D array of integers and its size as input and it removes all of the Armstrong numbers from this array. (May require shifting)

A positive integer is called an Armstrong number, if the sum of cubes of individual digits of number is equal to that number itself. For example

Number **153** is Armstrong because sum of cubes of individual digits $(1 * 1 * 1 + 5 * 5 * 5 + 3 * 3 * 3) = 153$

Number **12** is not Armstrong because sum of cubes of individual digits $(1 * 1 * 1 + 2 * 2 * 2) = 9$

Solution:

```
#include <iostream>
using namespace std;
int main()
{
    int n = 12;

    int ans = 0;

    while (n != 0)
    {
        int x;
        x = (n % 10);
        cout << x;
        n /= 10;

        ans += x * x * x;

    }
    cout << endl << ans;
    if (ans == n)
        cout << "\nit is an armstrong!";
    else
        cout << "\nit is not an armstrong";
    return 0;
}
```

Array 5:

Given the array `nums` consisting of `2n` elements in the form `[x1, x2, ..., xn, y1, y2, ..., yn]`.

Return the array in the form `[x1, y1, x2, y2, ..., xn, yn]`.

Example 1:

Input: `nums = [2,5,1,3,4,7]`, `n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since `x1=2`, `x2=5`, `x3=1`, `y1=3`, `y2=4`, `y3=7` then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2]`, `n = 2`

Output: `[1,2,1,2]`

Solution:

```
int main()
{
    int nums[] = { 2,5,1,3,4,7 };
    int result[6]{};
    int j = 0, n = 3;
    for (int i = 0; i < n; i++)
    {
        result[j++] = nums[i];
        result[j++] = nums[i+n];
    }
    for (int i = 0; i < 6; i++)
        cout << result[i] << " ";
    return 0;
}
```

Array 6:

Given the array `nums`, for each `nums[i]` find out how many numbers in the array are smaller than it. That is, for each `nums[i]` you have to count the number of valid `j`'s such that `j != i` **and** `nums[j] < nums[i]`.

Example 1:

Input: `nums = [8,1,2,2,3]`

Output: `[4,0,1,1,3]`

Explanation:

For `nums[0]=8` there exist four smaller numbers than it (1, 2, 2 and 3).

For `nums[1]=1` does not exist any smaller number than it.

For `nums[2]=2` there exist one smaller number than it (1).

For `nums[3]=2` there exist one smaller number than it (1).

For `nums[4]=3` there exist three smaller numbers than it (1, 2 and 2).

Example 2:

Input: `nums = [6,5,4,8]`

Output: `[2,1,0,3]`

Example 3:

Input: `nums = [7,7,7,7]`

Output: `[0,0,0,0]`

Sol:

```
int main()
{
    int nums[] = { 8,1,2,2,3 }, res[5];
    int len = 5, count;
    for (int i = 0; i < len; i++)
    {
        count = 0;
        for (int j = 0; j < len; j++)
        {
            if (nums[j] < nums[i])
                count++;
        }
        res[i] = count;
    }

    for (int j = 0; j < len; j++)
    {
        cout << res[j] << " ";
    }

    return 0;
}
```

ARRAY 7

You are given two arrays `nums1` and `nums2` consisting of positive integers.

You have to replace all the 0's in both arrays with strictly positive integers such that the sum of elements of both arrays becomes equal.

Return the minimum equal sum you can obtain, or -1 if it is impossible.

Examples:

Input: `nums1 = [3,2,0,1,0]`, `nums2 = [6,5,0]`

Output: 12

Input: `nums1 = [2,0,2,0]`, `nums2 = [1,4]`

Output: -1

Solution:

```
#include <iostream>

int maxSumAfterOneZeroChange(int nums1[], int m, int nums2[], int n) {
    int s1 = 0, s2 = 0;
    int c1 = 0, c2 = 0;
    for (int i = 0; i < m; i++) {
        s1 += nums1[i];
        c1 += (nums1[i] == 0);
    }
    for (int i = 0; i < n; i++) {
        s2 += nums2[i];
        c2 += (nums2[i] == 0);
    }
    if (c1 == 0 && s1 < s2 + c2) return -1;
    if (c2 == 0 && s2 < s1 + c1) return -1;
    return max(s1 + c1, s2 + c2);
}

int main() {
    int nums1[] = {1, 0, 1, 0, 1};
    int m = 5;
    int nums2[] = {0, 1, 0, 1, 0};
    int n = 5;
    int output = maxSumAfterOneZeroChange(nums1, m, nums2, n);
    if (output != -1)
        cout << "Maximum sum after one zero change: " << output << endl;
    else
        cout << "Cannot change a zero to make a greater sum." << endl;
    return 0;
}
```

Functions

Function 1

```
# include <iostream>
using namespace std;
f_1();
f_2(int,int);
f_3();
f_4(int);

main()
{
    int x=6;           //Line 1
    f_1();             //Line 2
    f_3();             //Line 3
    f_4(x);            //Line 4
}
f_1()
{
    int x=7, int y=5;
    f_2(x,y);
}
f_2(int x, int y)
{
    cout<<"X is "<<x<<" and Y is "<<y<<" in function f_1()."<<endl;
    f_3();
}
f_3()
{
    static int x=5;
    cout<<"A static X found in one of the functions with the value
"<<x<<" "<<endl;
    x++;
    f_4(x);
}
f_4(int x)
{
    cout<<"Value of X in f_4() is "<<x<<" "<<endl;
}
```



```
X is 7 and Y is 5 in function f_2().  
A static X found in one of the functions with the value 5.  
Value of X in f_4() is 6.  
A static X found in one of the functions with the value 6.  
Value of X in f_4() is 7.  
Value of X in f_4() is 6.
```

Function 2

```
int mystery(int x, int n)
{
    int val;
    val = 1;
    if (n >= 0)
    {
        if (n % 3 > 1)
            val = val * x;
        else
            val = val * 2;
    }
    return val;
}

void main()
{
    cout << "The mysterious value is: " << mystery(10, 3);
}
```

Output questions:

1)

```
int main(){  
  
    int suite = 5 ;  
    switch ( suite ) ;  
    {  
        case 0+5 ;  
        cout<< "\nClub"  ;  
        case 1+5 ;  
        cout<< "\nDiamond"  ;  
    }  
    return 0;  
}
```

2)

```
int x=10;  
  
void main()  
{  
    int x=20;  
    cout<<::x++;  
    cout<<x+::x;  
}
```